Machine Learning I
Classification

•••

POLI3148 Data Science in Politics and Public Administration

Dr. Chen Haohan

What is Machine Learning?

Machine learning (ML) is a field of study in artificial intelligence concerned with the development and study of statistical algorithms that can learn from data and generalize to unseen data, and thus perform tasks without explicit instructions.

Advances in the field of deep learning have allowed neural networks to surpass many previous approaches in performance.

Source: Wikipedia

What do social scientist care about machine learning?

- Discover complex relationship in big data
- Automate data annotation tasks
- * Make causal inferences

Essentials of Machine Learning: What matters

- **Transform** data to make them machine-readable
- **Train** Machine Learning models
- **Evaluate** Machine Learning models
- **Interpret** Machine Learning models

This Lecture:
Machine Learning for **CLASSIFICATION**

Classification Model

Definition: A model whose prediction is **a class**. For example, the following are all classification models:

- A model that predicts an input sentence's language (French? Spanish? Italian?).
- A model that predicts tree species (Maple? Oak? Baobab?).
- A model that predicts the positive or negative class for a particular medical condition.
- *A model that predicts if an official can get promotion*
- *A model that predicts if a war will happen*
- *A model that predicts which candidate can win the election*

In contrast, ***regression models*** predict numbers rather than classes.

Source: Google Machine Learning

Classification Model: Binary versus Multi-class

- **Binary Classification:** Outcomes can only be one of two states, for example:
  - An official gets promotion or not
  - A war happens or not
  - A candidate wins or loses

- **Multi-Class Classification:** Outcomes can be in one of multiple states, for example:
  - An official gets promoted, demoted, or stay in the position
  - No conflict, a non-violent conflict, or a violent conflict
  - A Democratic, Republican, or Independent candidate wins

We focus on Binary Classification in this lecture

Running Example: Lee & Shih (2023)

PNAS RESEARCH ARTICLE | POLITICAL SCIENCES

Machine-learning analysis of leadership formation in China to parse the roles of loyalty and institutional norms

Jonghyuk Lee[a,1] and Victor C. Shih[b]

Edited by Jean Hong, University of Michigan, Ann Arbor, MI; received March 29, 2023; accepted September 27, 2023 by Editorial Board Member Margaret Levi

A thriving cottage industry has long tried to predict the selection outcomes of the Chinese leadership using qualitative judgments based on historical trends and elite interviews. This study contributes to the discourse by adopting machine-learning techniques to quantitatively and systematically evaluate the promotion prospects of Chinese high-ranking officials. By incorporating over 250 individual features of approximately 20,000 high-ranking positions from 1982 to 2020, this paper calculated predicted probabilities of promotion for the 19th Politburo members of the Communist Party of China. The rankings of the promotion probabilities can be used not only to identify candidates who would have traditionally advanced within the party's promotion norms but also to gauge Xi Jinping's personal favoritism toward specific individuals. Based on different specifications for positions and periods, we developed measurements to quantify candidates' levels of perceived loyalty and promotion eligibility. The empirical results demonstrated that the newly formed 20th Politburo Standing Committee was predominantly composed of loyalists who would not have risen to such positions under conventional promotion standards. We further found that, even within his circle of known allies, Xi Jinping did not opt for candidates with strong credentials. The findings of this study underscore the increasing emphasis on loyalty and the diminishing role of institutional norms in China's high-ranking selections.

Chinese politics | machine learning | authoritarian politics

Significance

Machine-learning applications have great potential to offer valuable insights into a country's political dynamics. This study represents an attempt to examine the promotion outcomes of Chinese high-ranking officials by adopting machine-learning techniques. For this purpose, we established a comprehensive database containing biographical and career information for 5,110 cadres of the Communist Party of China. By incorporating 250 individual features constructed from the database, our

Links: PNAS, Appendix, Code and data

Workflow

Data Input:

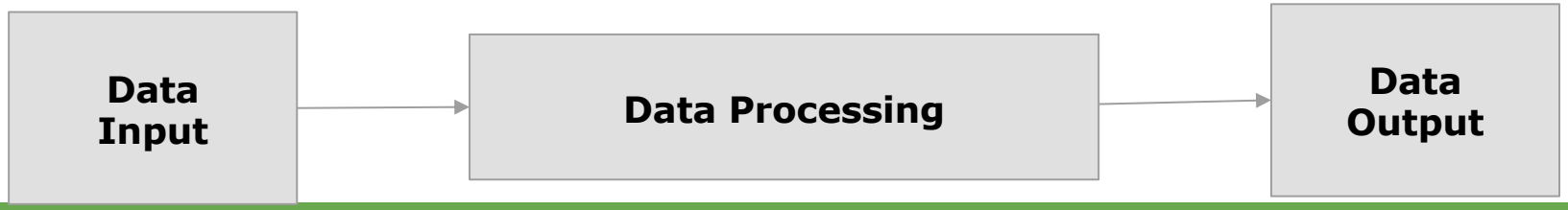
- Load data
- Define **X** and **y**
- Handle missing values
- Split data into training and test sets
- Re-formats for Machine Learning tool

Data Processing:

- Define model
- Train model

Data Output:

- Evaluate model
- Interpret model
- Discussion: What your results say about the substantive topics



Load Data: A Special Case -- R data format

```
!pip install pyreadr
import pandas as pd
import os
import pyreadr


# Get path to project folder
PATH_PROJECT = "[YOUR PATH TO PROJECT FOLDER]"
# Set project folder to working directory
os.chdir(PATH_PROJECT)

# Read data
df = pyreadr.read_r('data_raw/data_train_2020_DEC.RData')

# The function read the as a dictionary of pandas dataframe. We need to first check the names of
dataframes and then extract what we need.
df.keys()

# The name of the dataset is "data_training". Use this as the dictionary key
df = df['data_training']
```

Load Data: What the data is like

```
# Print the dataframe
df
```

	promo	pc	year	posi	age	age2	age60over	age61over	age62over	age63over	...	PAT_POLI_all5_promo_cur_dum	PAT_POLI_all_stay_cur_dum	PAT_POLI_all5_stay_cur_dum	PAT_POLI_all_remova
0	0.0	15.0	1997.0	Center	55.0	3025.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	
1	0.0	17.0	2007.0	Center	65.0	4225.0	1.0	1.0	1.0	1.0	...	0.0	0.0	0.0	
2	0.0	16.0	2002.0	Center	60.0	3600.0	0.0	0.0	0.0	0.0	...	0.0	1.0	1.0	
3	0.0	18.0	2012.0	Center	61.0	3721.0	1.0	0.0	0.0	0.0	...	0.0	1.0	1.0	
4	0.0	17.0	2007.0	Center	56.0	3136.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
20134	0.0	19.0	2020.0	Prov	58.0	3364.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	
20135	0.0	19.0	2020.0	Prov	59.0	3481.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	
20136	0.0	19.0	2020.0	Prov	57.0	3249.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	
20137	0.0	19.0	2020.0	Prov	49.0	2401.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	
20138	0.0	19.0	2020.0	Prov	60.0	3600.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	

20139 rows × 251 columns

Identify X and y

y: namely, outcome/ target/ dependent variable

X: namely, oredictors, features, independent variables

Read the codebook

```
df.columns
y_name = ['promo']
X_names = ['age', 'gender', 'ethnic', 'edu']
df_s = df.loc[:, y_name + X_names]
```

Handle Missing Values

Missing values can **cause problem** with your machine modeling. Handle them before passing your data to machine learning models

We use a simplest approach -- removing rows that contains missing values. Alternatively, you may use *imputation* to fill in missing values.

```
df_s.isnull().sum()
```

```
# Quick fix: Drop all rows that contain NAs
df_s_dropna = df_s.dropna()
df_s_dropna
```

Turn your X and y into Numpy Arrays

- Turn your data into a format that is Sklearn can process efficiently (and with lower likelihood of errors)
- Numpy arrays

To add to the “preamble” chunk where you load all the libraries

```
import numpy as np
```

Convert X and y to numpy arrays

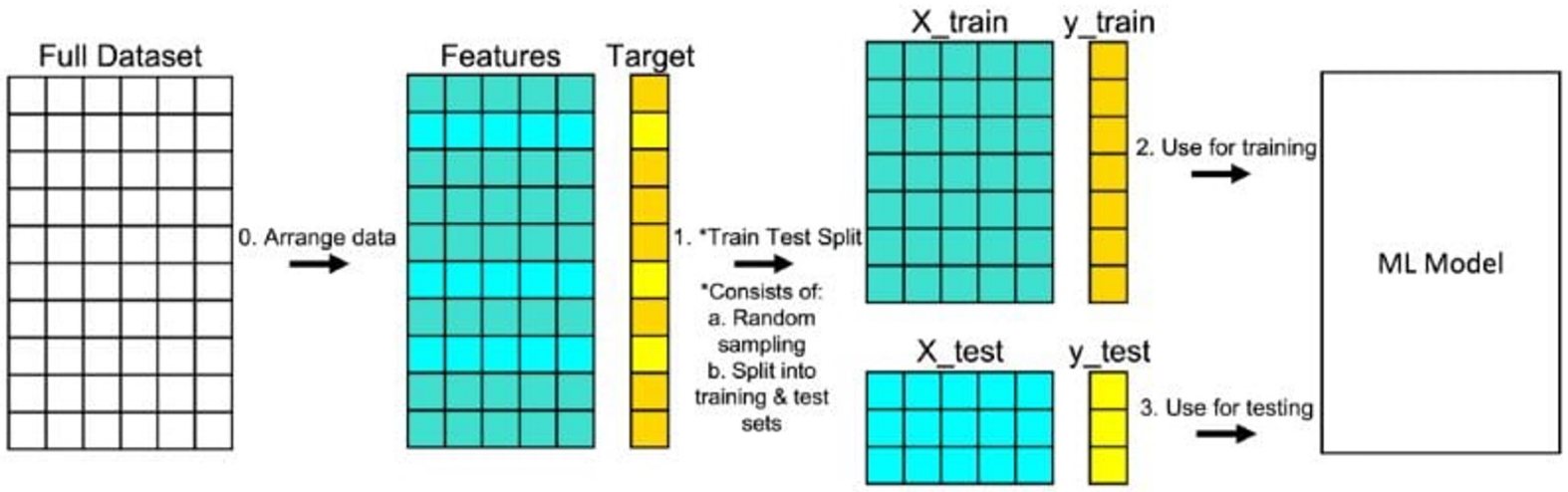
```
X = df_s_dropna.loc[:, X_names].to_numpy()
y = df_s_dropna.loc[:, y_name].to_numpy()
y = y[:, 0]
```

What does this do?

Check your constructed data:

- No missing value
- All columns are numeric
- Keep **x_names** for future use

Split Data into Training and Test Sets: How, conceptually



Split Data into Training and Test Sets: How, technically

Use a function from sklearn called “train_test_split”

Add the following line to the preamble

20% of the data are put in a “held-out” test set

Then run the following code

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2,
random_state = 123)
```

A random seed that controls the random split.

Pick your own *lucky number.*

Check the split data (as a sanity check)

```
print("X_train")
print(X_train)
print(X_train.shape)

print("------------")

print("X_test")
print(X_test)
print(X_test.shape)
```

```
print("y_train")
print(y_train)
print(y_train.shape)

print("------------")

print("y_test")
print(y_test)
print(y_test.shape)
```

Split Data into Training and Test Sets: Why?

- A key objective of Machine Learning is to perform **prediction**
- We want to know ***how well*** a model performs if it is given **new** data it ***has not seen*** before
- E.g., in our case, we are interested in knowing how well our Model can take the profile of a ***new*** official and predict how likely they will get promoted

Split Data into Training and Test Sets: Why?

How do we get an idea of **“how well?”**

- **Hide** a part of the data (X_test, y_test) from the Machine Learning model
- **Train** the Machine Learning model using only a part of our data (X_train, y_train)
- **Apply** the Machine Learning model to the “**hidden”** data (X_test) to get predictions (y_test_predicted), pretending we don’t know the answer
- **Compare** Machine Learning model’s **predictions** (y_test_predicted) with the **answer** we already know (y_test). Their discrepancy tells us how well the model does

This works like a *mock-test*.

Consequences of not splitting the data: “Over-fitting”

Split Data into Training and Test Sets: Why?

The biggest consequence of training and evaluating the model in this manner:

Overfitting

Overfitting means creating a model that matches (memorizes) the training set so closely that the model fails to make correct predictions on new data.

Source: Google Machine Learning (check out this resource!)

Workflow

Data Input:

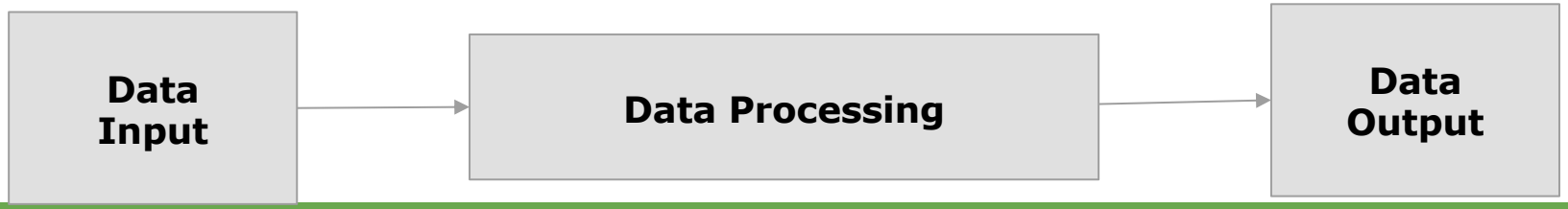
- Load data
- Define **X** and **y**
- Handle missing values
- Split data into training and test sets
- Re-formats for Machine Learning tool

Data Processing:

- Define model
- Train model

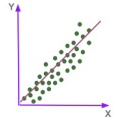
Data Output:

- Evaluate model
- Interpret model
- Discussion: What your results say about the substantive topics



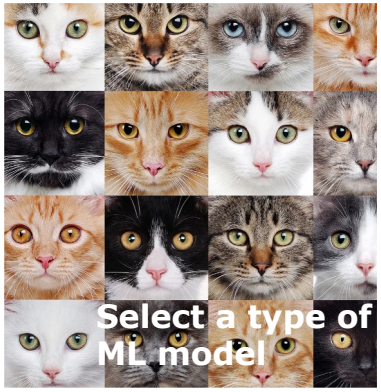
What does “defining” and “training” an ML model mean?

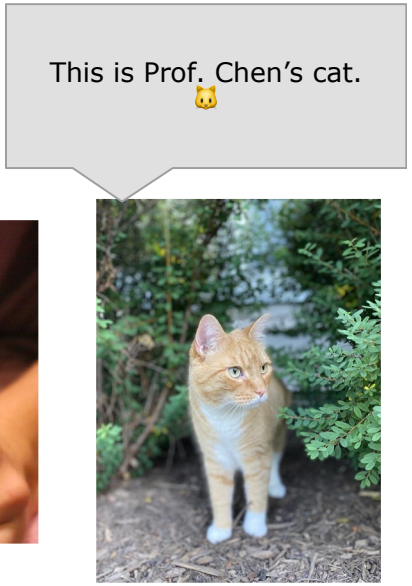
- An ML model has an “architecture” (and is named after it)
  - Mathematic equations specifying the relationship between **data** and **parameters**
  - Engineering design on how to learn from the **data** to update the **parameters**, so that the model reflects the patterns of the **data**
  - Simplest example: y = ax + b (“Linear Regression”)



- First, ***select*** the type (“architecture”) of model to use
- Second, ***initialized*** the model, with parameters randomly assigned
- Third, ***train*** the model with data to update the parameters

Defining and Training a Machine Learning Model





But, how do these models work?

- We don’t have time to discuss the “architecture” of the variety of ML models
- Focus on how to use these models in a advisable workflow with Python
- My requirement: When using a ML model
  - **Must:** Refer to models by its names properly
  - **Must:** Report relevant information properly
  - **Strongly recommended:** Attempt multiple models for the same data
  - **Preferred:** Understand the intuition of the architecture of the models you use

Tool: scikit-learn 

https://scikit-learn.org/stable/

- THE most popular Python library for **Machine Learning**

  So popular that you can write it on your resume!

- We previously used it for text analysis
  - Constructing “Bag-of-word” tables
  - Train topic modeling using the “Latent Dirichlet Allocation” function
- **Advantage:** One standardized workflow for vavious types of ML models

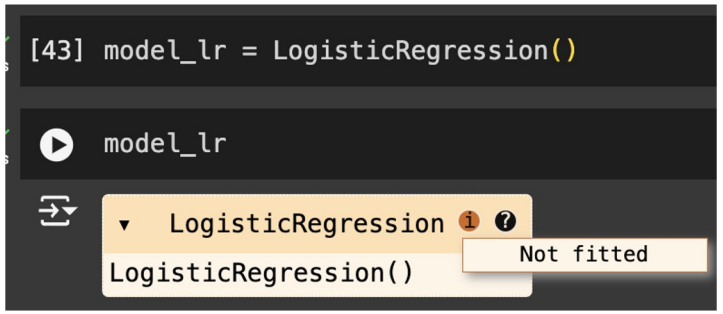
Classification Models with Sklearn

- An overview of Machine Learning models available with Sklearn: https://scikit-learn.org/stable/supervised_learning.html
- Models we will discuss
  - Logistic Regression
  - Decision Tree
  - Random Forest

Other machine learning models you may explore: Decision Tree, Support Vector Machines, Gradient Boosting

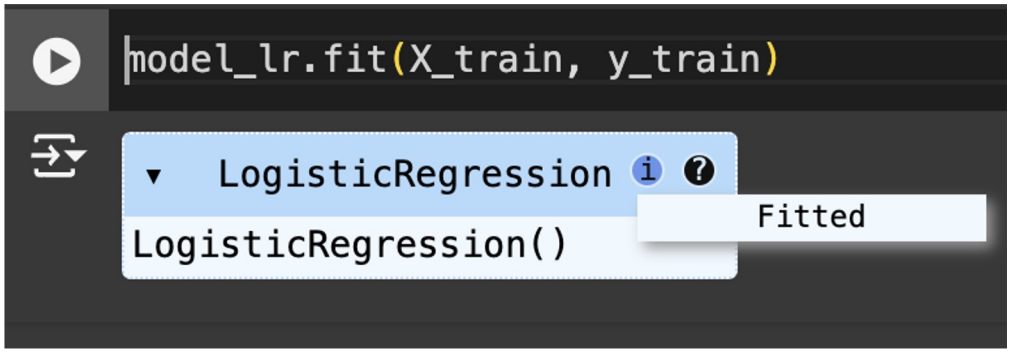
Logistic Regression: Define Model

```
from sklearn.linear_model import LogisticRegression

model_lr = LogisticRegression()
```



Logistic Regression: Train Model

model_lr.fit(X_train, y_train)



Workflow

Data Input:

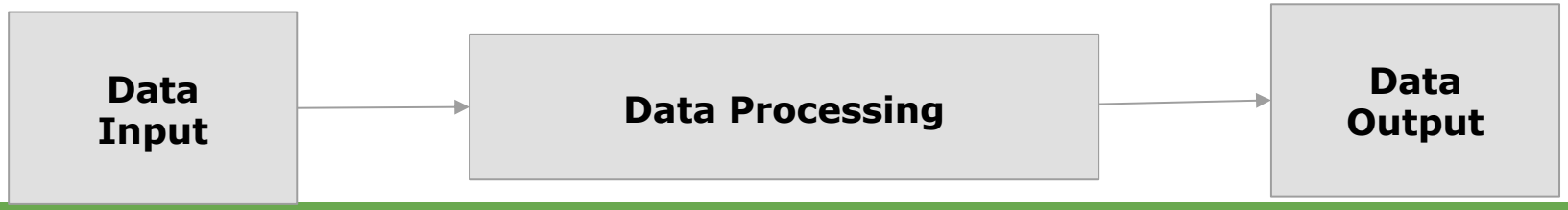
- Load data
- Define **X** and **y**
- Handle missing values
- Split data into training and test sets
- Re-formats for Machine Learning tool

Data Processing:

- Define model
- Train model

Data Output:

- Interpret model
- Evaluate model
- Discussion: What your results say about the substantive topics



Interpret Model

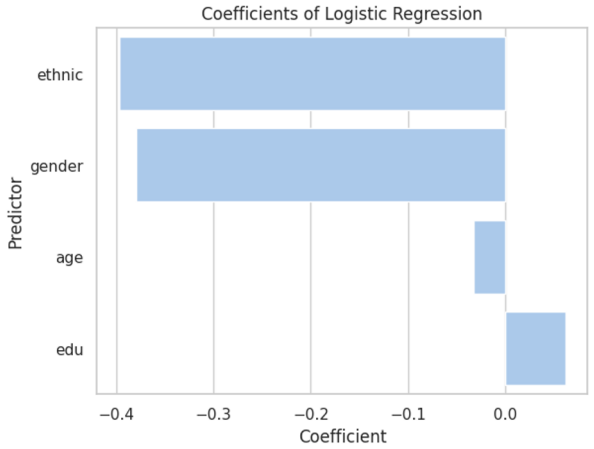
Interpreting the ML model helps us understand the association between **X** and **y**

- Not all ML models are *interpretable* (in general, models with more complex architecture are less interpretable)
- Interpretable ML models need to be interpreted in different ways
- Logistic Regressions are interpretable

  *We can understand how each column in **X** is associated with **y***

Interpret Model: Logistic Regression

```
model_interpretation = pd.DataFrame({'Predictor': X_names, 'Coefficient': model_lr.coef_[0]})

model_interpretation = model_interpretation.sort_values('Coefficient')

model_interpretation

sns.barplot(y = "Predictor", x = "Coefficient", data = model_interpretation)

plt.title("Coefficients of Logistic Regression")
```



Evaluate Model

- Evaluating a ML model tells us how well it perform with new data that it has never seen
- In our example, for officials whose profiles our model has never seen, how well does it predict their odds of promotion
- Make use of **X_test** and **y_test**

Evaluate Classifier 1: Make Prediction with New Data

Gets the predicted probability of two classes (Negative and Positive)

y_test_predicted_probability = model_lr.predict_proba(X_test)

y_test_predicted_probability

```
array([[0.91362517, 0.08637483],
       [0.92097201, 0.07902799],
       [0.91123255, 0.08876745],
       ...,
       [0.89447852, 0.10552148],
       [0.91104186, 0.08895814],
       [0.94051761, 0.05948239]])
```

y_test_predicted_probability_1 = y_test_predicted_probability[:, 1]

y_test_predicted_probability_1

```
array([0.08637483, 0.07902799, 0.08876745, ..., 0.10552148, 0.08895814,
       0.05948239])
```

Evaluate Classifier 1: Binarize the Output – Select a Threshold

y_test_predicted = (y_test_predicted_probability_1 > 0.1)

This tells the machine: When the predicted probability of the positive case (e.g., being promoted) is greater to 0.1, I consider it as a **positive** prediction

Evaluate Classifier 2: Compare the Predicted and Actual Outputs

Get a **Confusion Matrix**

from sklearn import metrics
metrics.confusion_matrix(y_test, y_test_predicted)

```
array([[3067,  540],
       [ 267,   78]])
```

```
([[2850,  768],
  [ 226,  108]])
```

		Predicted **y**	
		Negative Not promoted	**Positive** promoted
Actual **y**	**Negative** Not promoted	✅ True Negative	❌ False Positive
	Positive promoted	❌ False Negative	✅ True Positive

Evaluate Classifier 2: Compare the Predicted and Actual Outputs

Get individual count numbers

```
true_negative, false_positive, false_negative, true_positive = metrics.confusion_matrix(y_test, y_test_predicted).ravel()

print(f"True Negative: {true_negative}")

print(f"False Negative: {false_negative}")

print(f"True Positive: {true_positive}")

print(f"False Positive: {false_positive}")
```

```
True Negative: 3067
False Negative: 267
True Positive: 78
False Positive: 540
```

Why so complicated? Can't we have just one number

The complexity with evaluating a classification model

- A perfect ML model: All predictions true positive and true negative
- In reality: Trade-off
  - Increase true positive → increase false positive
  - Increase true negative → increase false negative
- Different applications are more concerned about different types of error

  For example, if doctors develop a ML model for diagnostic, false negative results will be very costly

- Data scientist develop a set of evaluation metrics to summarize the performance of an ML model along different dimensions

Evaluation Metrics

Terms: “TP” = True positive; “FP” = False positive; “TN” = True negative; “FN” = False negative

- **Accuracy**: the prop. of all classifications that were correct
  (TP + TN) / (TP + TN + FP + FN)
- **Precision**: the prop. of all the model's positive classifications that are actually positive
  TP / (TP + FP)
- **Recall** (True Positive Rate): the prop. of all actual positives that were classified correctly as positives
  TP / (TP + FN)
- * False Positive Rate: the prop. of all actual negatives that were classified incorrectly as positives
  FP / (FP + TN)
- **F1 score** (“average” of precision and recall)
  2 * (Precision * Recall) / (Precision + Recall)

Read further: Google Machine Learning

Let's calculate it manually, for once

Use the formula to calculate them.

accuracy = (true_positive + true_negative) / (true_positive + true_negative + false_positive + false_negative)

precision = true_positive / (true_positive + false_positive)

recall = true_positive / (true_positive + false_negative)

fpr = false_positive / (false_positive + true_negative)

f1 = 2 * (precision * recall) / (precision + recall)

Sklearn provides functions to make the calculation convenient

```
from sklearn import metrics # add to preamble and run

print(metrics.accuracy_score(y_test, y_test_predicted))

print(metrics.precision_score(y_test, y_test_predicted))

print(metrics.recall_score(y_test, y_test_predicted))

print(metrics.f1_score(y_test, y_test_predicted))
```

This should be the **binarized (1/0)** output, not the probabilities!

Choice of Threshold can change all these metrics

y_test_predicted = (y_test_predicted_probability_1 > 0.1)

Change to 0.08, 0.1, 0.12, 0.15, … 0.5

Challenge:

Write a function, see how changes in thresholds alter the evaluation metrics

Towards A More Comprehensive Measure of Performance: The ROC Curve

Evaluate a model's quality across all possible thresholds

Receiver-operating characteristics curve (ROC curve)

Idea:

- Change the *thresholds* from 0 to 1
- Get the combination of True Positive Rate (Precision) and False Positive Rate at each threshold
- Plot them (X = FPR, Y = TPR). This is called the **ROC curve**
- Advanced: The Area Under the ROC curve (AUC) measures the model's performance

Intuitively, AUC tells us, with the *trade-off* between true and false positive, how well can the model do

Further Reading: Google Machine Learning

Towards A More Comprehensive Measure of Performance: The ROC Curve

```
auc = metrics.roc_auc_score(y_test, y_test_predicted_probability_1)

false_positive_rate, true_positive_rate, _ = metrics.roc_curve(y_test, y_test_predicted_probability_1)
```

```
import seaborn as sns

from matplotlib import pyplot as plt

plt.figure()
plt.plot(false_positive_rate, true_positive_rate)
plt.plot([0, 1], [0, 1], '--', color = 'gray')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title(f'ROC Curve (AUC = {round(auc, 2)})')
plt.show()
```

Finally: Discussion of the ML Results

- Evaluating the model

  How well does the model do in predicting new data?

- Interpreting the model

  What does the parameters say about the associations in your data?

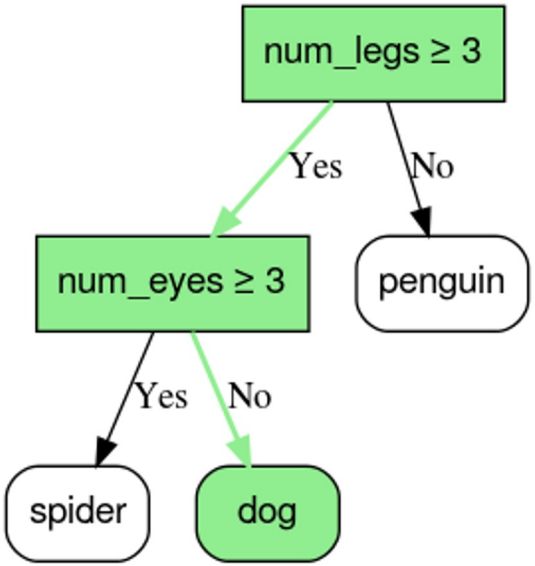
We have now done a round of standard ML workflow with Logistic Regression, the simplest possible ML model!

Other models?

Same workflow for data preparation, training, and evaluation. **Different** way to interpret.

A Different Family of ML Model: Decision Trees

Intuition of Decision Trees for classification:
The machine do classification using a decision tree 🌲

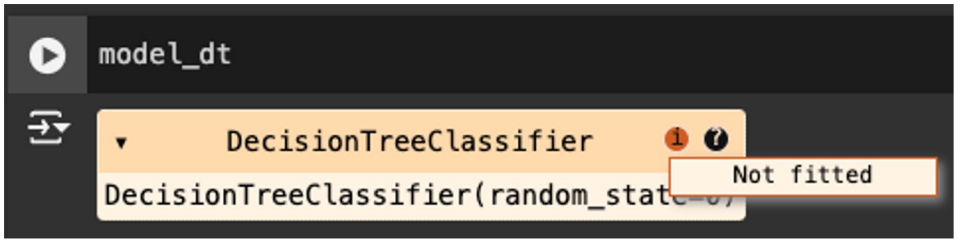


Penguin, spider, or dog?

Define a Decision Tree Classifier

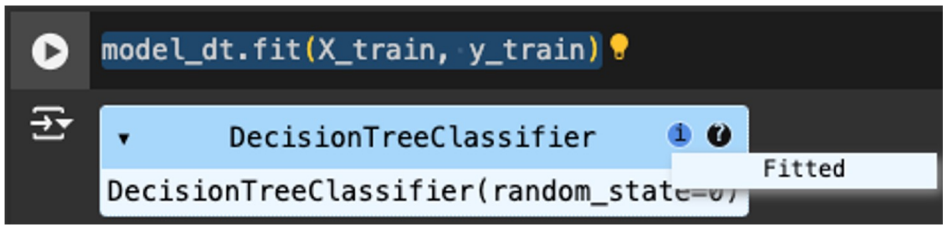
from sklearn import tree

model_dt = tree.DecisionTreeClassifier(random_state=0)



Train a Decision Tree Classifier

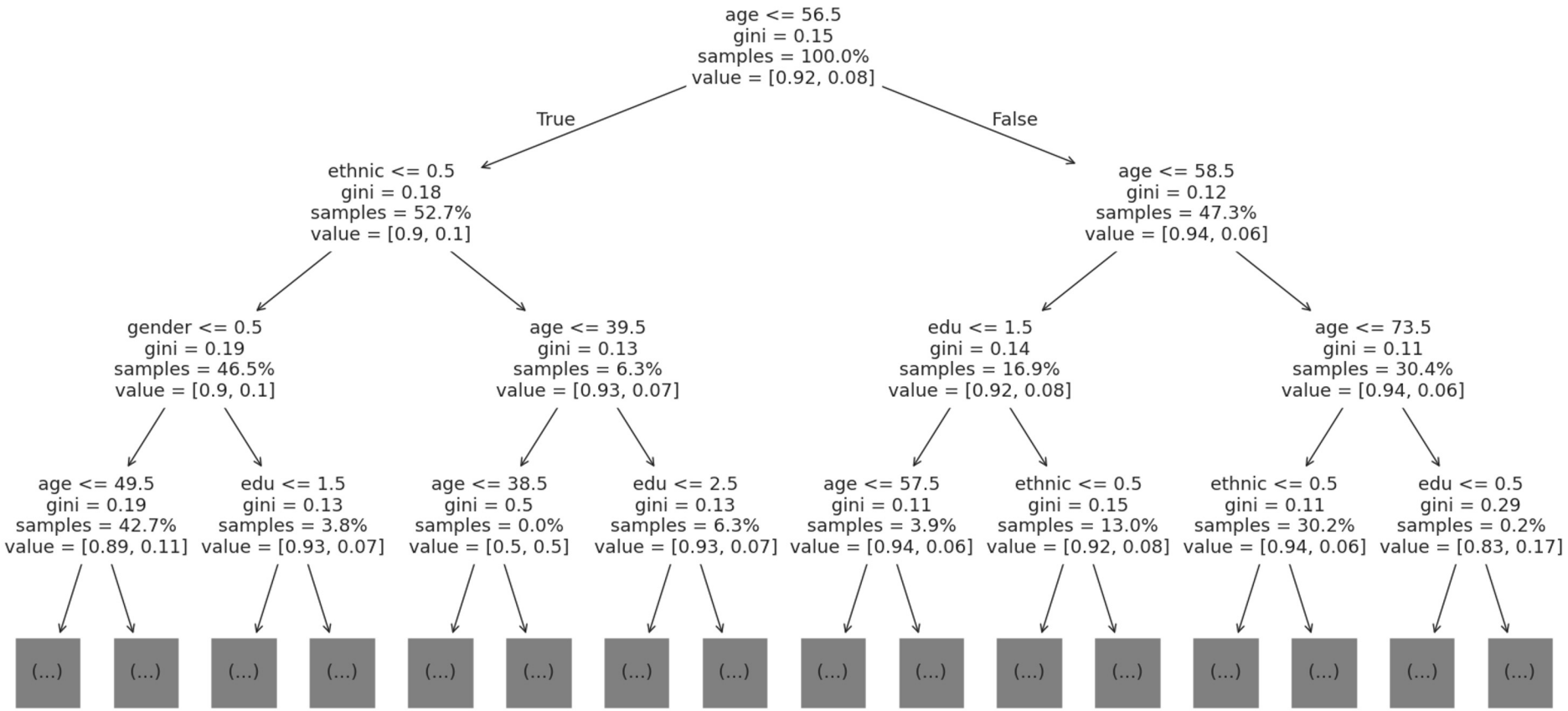
model_dt.fit(X_train, y_train)

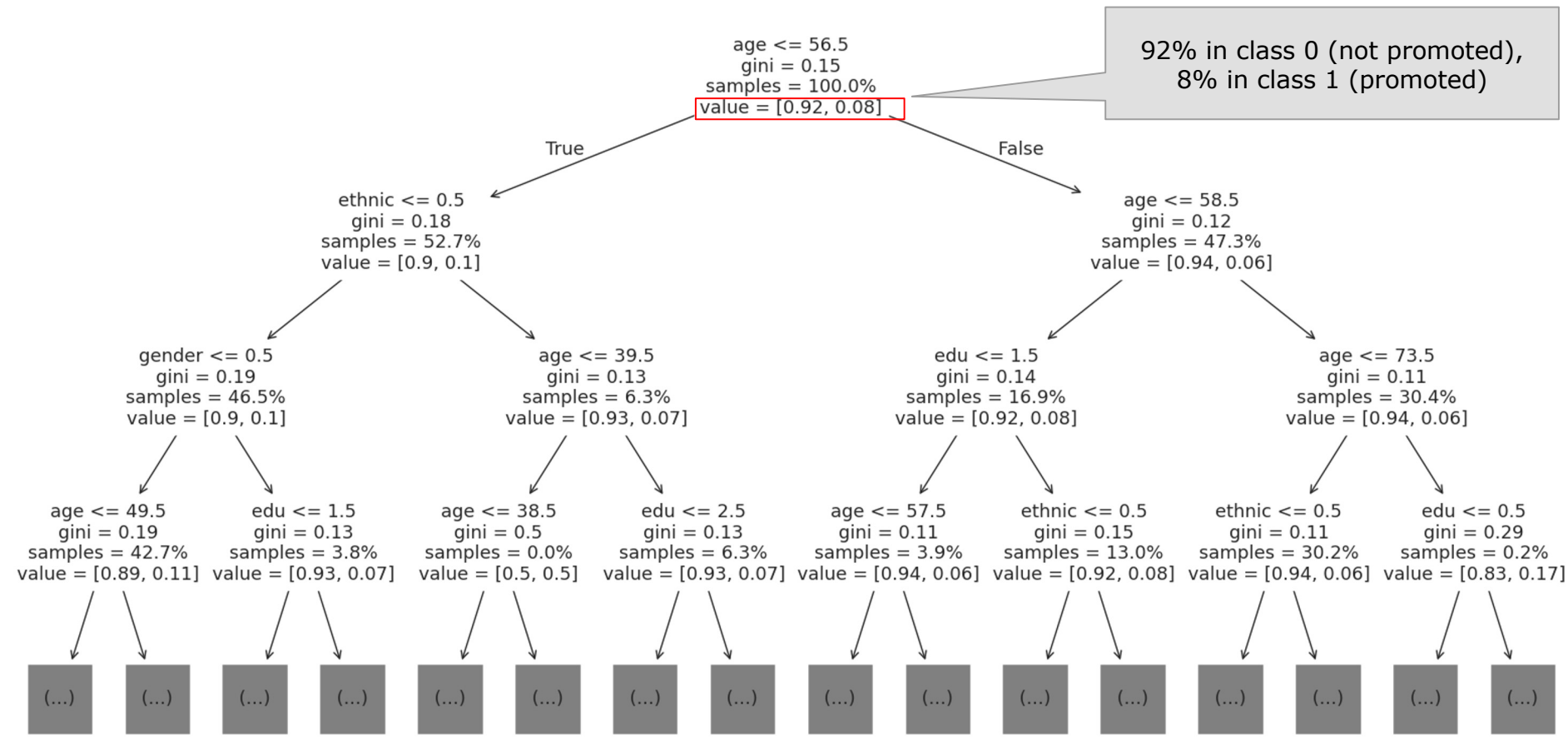


Visualize the Decision Tree

```
plt.figure(figsize = (20, 10))
tree.plot_tree(model_dt, feature_names = X_names,
               max_depth = 3, fontsize = 13, proportion = True, precision = 2)
plt.show()
```

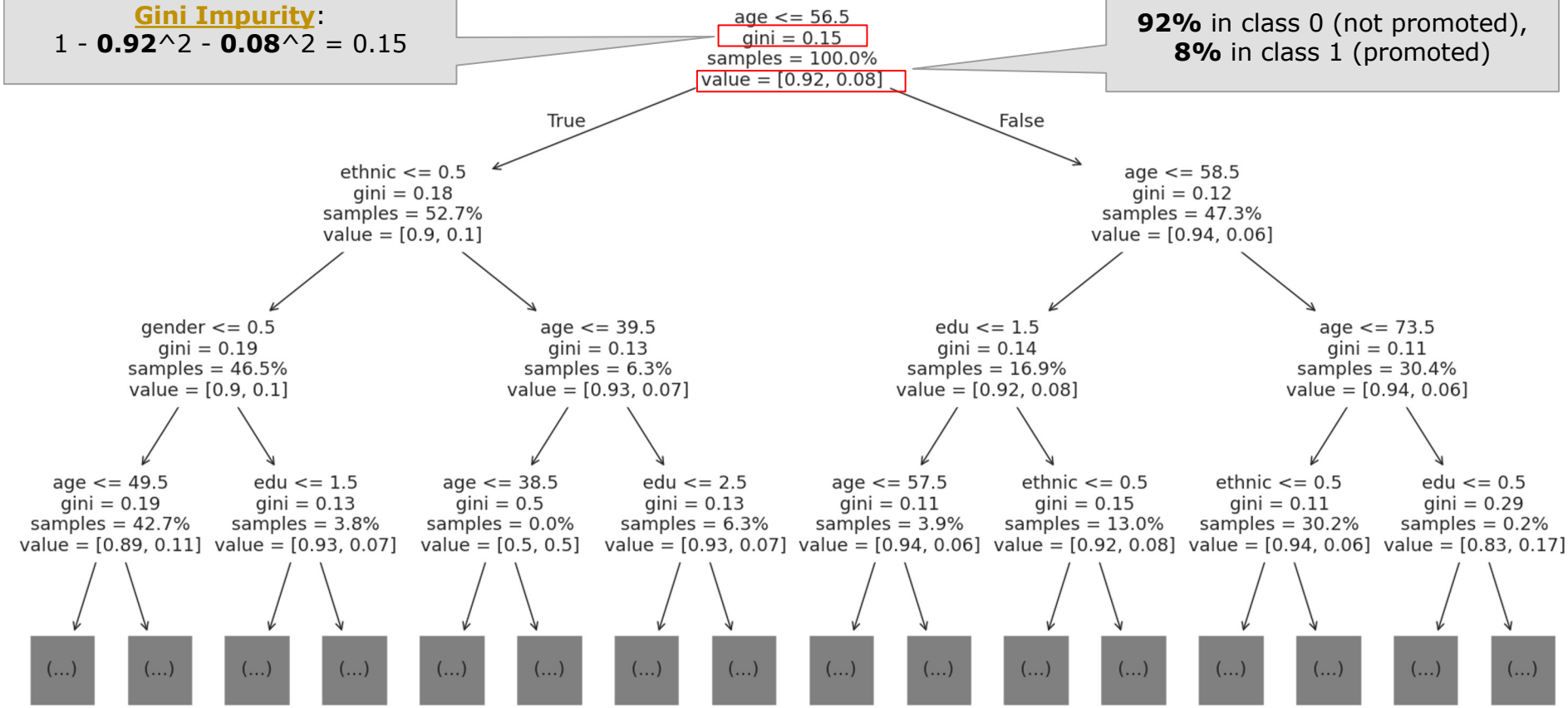
Further Reading: plot_tree documentation


age <= 56.5
gini = 0.15
samples = 100.0%
value = [0.92, 0.08]
True
False
ethnic <= 0.5
gini = 0.18
samples = 52.7%
value = [0.9, 0.1]
age <= 58.5
gini = 0.12
samples = 47.3%
value = [0.94, 0.06]
gender <= 0.5
gini = 0.19
samples = 46.5%
value = [0.9, 0.1]
age <= 39.5
gini = 0.13
samples = 6.3%
value = [0.93, 0.07]
edu <= 1.5
gini = 0.14
samples = 16.9%
value = [0.92, 0.08]
age <= 73.5
gini = 0.11
samples = 30.4%
value = [0.94, 0.06]
age <= 49.5
gini = 0.19
samples = 42.7%
value = [0.89, 0.11]
edu <= 1.5
gini = 0.13
samples = 3.8%
value = [0.93, 0.07]
age <= 38.5
gini = 0.5
samples = 0.0%
value = [0.5, 0.5]
edu <= 2.5
gini = 0.13
samples = 6.3%
value = [0.93, 0.07]
age <= 57.5
gini = 0.11
samples = 3.9%
value = [0.94, 0.06]
ethnic <= 0.5
gini = 0.15
samples = 13.0%
value = [0.92, 0.08]
ethnic <= 0.5
gini = 0.11
samples = 30.2%
value = [0.94, 0.06]
edu <= 0.5
gini = 0.29
samples = 0.2%
value = [0.83, 0.17]
(...)
(...)
(...)
(...)
(...)
(...)
(...)
(...)
(...)
(...)
(...)
(...)
(...)
(...)
(...)
(...)


age <= 56.5
gini = 0.15
samples = 100.0%
value = [0.92, 0.08]
92% in class 0 (not promoted),
8% in class 1 (promoted)
True
False
ethnic <= 0.5
gini = 0.18
samples = 52.7%
value = [0.9, 0.1]
age <= 58.5
gini = 0.12
samples = 47.3%
value = [0.94, 0.06]
gender <= 0.5
gini = 0.19
samples = 46.5%
value = [0.9, 0.1]
age <= 39.5
gini = 0.13
samples = 6.3%
value = [0.93, 0.07]
edu <= 1.5
gini = 0.14
samples = 16.9%
value = [0.92, 0.08]
age <= 73.5
gini = 0.11
samples = 30.4%
value = [0.94, 0.06]
age <= 49.5
gini = 0.19
samples = 42.7%
value = [0.89, 0.11]
edu <= 1.5
gini = 0.13
samples = 3.8%
value = [0.93, 0.07]
age <= 38.5
gini = 0.5
samples = 0.0%
value = [0.5, 0.5]
edu <= 2.5
gini = 0.13
samples = 6.3%
value = [0.93, 0.07]
age <= 57.5
gini = 0.11
samples = 3.9%
value = [0.94, 0.06]
ethnic <= 0.5
gini = 0.15
samples = 13.0%
value = [0.92, 0.08]
ethnic <= 0.5
gini = 0.11
samples = 30.2%
value = [0.94, 0.06]
edu <= 0.5
gini = 0.29
samples = 0.2%
value = [0.83, 0.17]
(...)

Gini Impurity:
1 - **0.92**^2 - **0.08**^2 = 0.15

92% in class 0 (not promoted),
8% in class 1 (promoted)



Interpret the Model: Feature Importance

- **Gini Impurity**: the intuition
  - How "*impure*" is a branch.
  - Desired: a *pure* branch – everything under a branch is 1 (promoted) or 0 (not promoted)
  - Ex: Age <= 56, ethnic minority, female, princelin ⇒ ALL gets promoted! (pure)
  - Implausible and undesirable in real data analysis.
    - There are exceptions, however well the tree is built
    - Want to avoid over-fitting
- ⇒ Model attributes of interest: **Feature Importance**

  Measures how much a predictor reduces **impurity**

Interpret Model: Obtain and Visualize Feature Importance

```
model_interpretation = pd.DataFrame({"Predictor": X_names, "Importance":
model_dt.feature_importances_})

plt.figure(figsize = (6, 4))

sns.barplot(y = "Predictor", x = "Importance", data = model_interpretation)

plt.title("Feature Importance of Decision Tree")
```

Evaluate Model

Change the threshold and see what happens

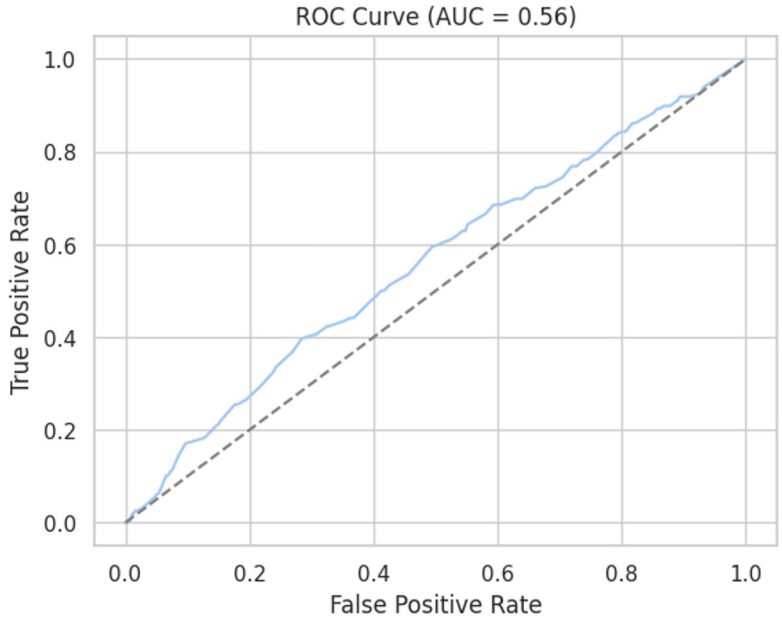
```
THRESHOLD = 0.1

# Get predicted probability for the test set

y_test_predicted_probability = model_dt.predict_proba(X_test)

y_test_predicted_probability

y_test_predicted_probability_1 = y_test_predicted_probability[:, 1]

y_test_predicted = (y_test_predicted_probability_1 > THRESHOLD)

# Calculate Accuracy

print(f"Accuracy: {metrics.accuracy_score(y_test, y_test_predicted)}")

print(f"Precision: {metrics.precision_score(y_test, y_test_predicted)}")

print(f"Recall: {metrics.recall_score(y_test, y_test_predicted)}")

print(f"F1 Score: {metrics.f1_score(y_test, y_test_predicted)}")

print(f"ROC-AUC: {metrics.roc_auc_score(y_test, y_test_predicted_probability_1)}")
```

```
Accuracy: 0.6219635627530364
Precision: 0.10461114934618032
Recall: 0.4405797101449275
F1 Score: 0.16907675194660735
ROC-AUC: 0.5585339295974413
```

Evaluate Model: ROC Curve

```
false_positive_rate, true_positive_rate, _ = metrics.roc_curve(y_test, y_test_predicted_probability_1)
auc = metrics.roc_auc_score(y_test, y_test_predicted_probability_1)
```

```
plt.figure()
plt.plot(false_positive_rate, true_positive_rate)
plt.plot([0, 1], [0, 1], '--', color = 'gray')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title(f'ROC Curve (AUC = {round(auc, 2)})')
plt.show()
```



Many Trees: Random Forest

Random Forest Model

An "Ensemble" model

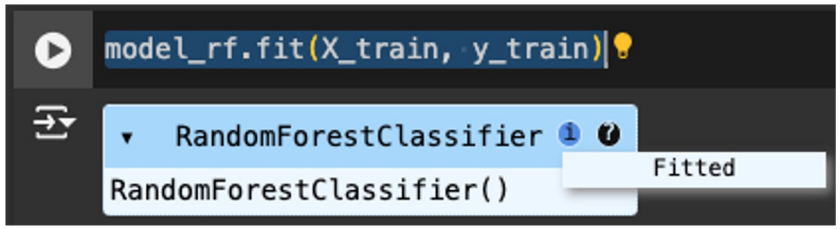
Grow many "Decision Trees" → Let the trees "vote" for the best outcome

- Each Decision Tree is train with *a different variant* of the training data
  - A random sample of the rows ("bootstrapping")
  - A random sample of columns/ features

- A more complex and powerful model compared to Logistic Regression and Decision Trees
- Widely used

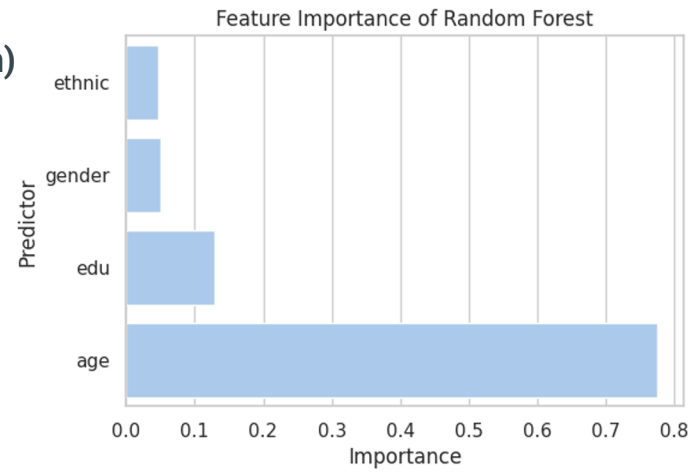
Define and Train a Random Forest Model

```
from sklearn.ensemble import RandomForestClassifier

model_rf = RandomForestClassifier(n_estimators = 100) # 100 trees

model_rf.fit(X_train, y_train)
```



Interpret a Random Forest Model

```
model_interpretation = pd.DataFrame({"Predictor": X_names, "Importance": model_rf.feature_importances_})

model_interpretation = model_interpretation.sort_values("Importance")


plt.figure(figsize = (6, 4))

sns.barplot(y = "Predictor", x = "Importance", data = model_interpretation)

plt.title("Feature Importance of Random Forest")
```



Evaluate a Random Forest Model

Same as Logistic Regression and Decision Trees